
UNIT 10 OBJECT DESIGN

Structure	Page No.
10.0 Introduction	
10.1 Objectives	
10.2 Object Design for Processing	
10.3 Object Design Steps	
10.4 Create Links between High-Level Requirements to Low-Level Services	
10.5 Realisation of Use Cases Using Operations	
10.6 Designing Algorithms	
10.6.1 Choosing Algorithms	
10.6.2 Selecting Data Structure	
10.6.3 Defining Internal Classes and Operations	
10.6.4 Assigning Operations to Classes	
10.7 Design Optimisation	
10.7.1 Look for Efficient Access Paths	
10.7.2 Rearranging Computing Tasks for Greater Efficiency	
10.7.3 Saving Intermediate Results for Avoiding Re-computation	
10.8 Implementation of Control	
10.8.1 Find the Respective Classes for States	
10.8.2 Find the Triggering Event	
10.8.3 Merge State Diagrams into Code	
10.8.4 Examining Against Other State Diagrams	
10.8.5 Examining Against Class Model	
10.8.6 Examining Against Sequence Diagram	
10.9 Adjustment of Inheritance	
10.9.1 Rearranging Classes and Operations	
10.9.2 Abstraction of Common Behavior	
10.10 Design of Associations	
10.11 Summary	
10.12 Solutions/Answers to Check Your Progress	
10.13 References/Further Readings	

10.0 INTRODUCTION

In the analysis phase of system development, we had created a blueprint which had helped us move ahead and decide what a programmer in implementation should do. Object design aims to map the layout created in the analysis phase to actual definitions of classes and their associations, followed by choice of corresponding algorithms for operations. So, in object design, we work on resolving details and repeating the decomposition at lower levels of abstraction.

The benefit of using object-oriented methodology is that it applies equally well in the domains of real-world mapping specifications and computer-based implementations. When the class design seems to be ready at one level of abstraction, it means now we need to iterate the process for lower levels.

This unit covers the aspects of object design on realising use cases, designing algorithms, design optimisation and adjustment of the inheritance hierarchy.

10.1 OBJECTIVES

After going through this Unit, you should be able to :

- Describe steps of object design,
- Use algorithms that minimise costs,
- Selection of data structures for algorithms,
- Defining new classes and operations,
- Allocating responsibility for operation to suitable classes, and

10.2 OBJECT DESIGN FOR PROCESSING

The analysis model provides high-level operations, so the system has a robust and stable structure. However, the model is informal and cannot be used to move toward the implementation(source code). Subsequently, as we approach the class model in the design phase, it advocates the addition of implementation details. The analysis model assumes that all resources are available for implementation as they would be in an ideal world. But a reality check indicates that the analysis model needs to be adapted to the environment in which it gets implemented.

The class design must be restructured for efficiency, for creating internal data structures and algorithms, consider performance requirements, real-time requirements and concurrency, properties of programming languages and so on. This opens up the possibility of having entirely new classes. But for beginners, the best approach starts with incorporating the analysis classes directly from the analysis model to the design model. With this, essential features like traceability and consistency in the system to be developed are easy to obtain as the models evolve.

The analysis classes are realised during design so that the execution time is minimised, space complexity is optimised, and other cost measures are handled. In this process, new operations may arise. Also, the complex operations have to be broken into simpler operations. This decomposition is iterative and keeps repeating for subsequent lower levels of abstractions. New classes for handling distributed operations or storing intermediate results may be required. But one must be cautious while adding new classes; designers should avoid over-optimisation, taking care of ease of maintainability and extensibility. Hence analysis model is built when the problem is investigated, while the design is about inventing a solution in the computer domain.

To put it more concisely:

“Analysis = What”;

“Design = How”

The design model is driven by the requirement to produce a finished good, and outcomes are dependent on the experience of the team, reuse opportunities and personal preferences of the designer. The designer studies both the requirements and analysis artifacts to produce an effective solution.

During the design, there are certain technology choices to be made. These choices influence libraries, patterns and frameworks that can be deployed in the model. The experts give preference to making the design more general rather than being dependent on technology. Object-oriented software development is incremental, and the entire system cannot be built in a single iteration. Hence, prioritised use cases should be designed initially. So, in the next section, we learn how to design objects.

10.3 OBJECT DESIGN STEPS

For designing objects, the following steps are involved:

- Creating links between high-level requirements to low-level services.
- Realisation of use-cases using operations.
- Formulating an algorithm
- Designing operations that assist high-level methods.
- Refactoring for a clean design.
- Optimisation of data access paths.
- Realise behaviour that must be deployed.
- Adjustment of classes to specify inheritance.
- Organise classes with their associations.

These steps have been discussed in detail in the subsequent sections.

10.4 CREATE LINKS BETWEEN HIGH-LEVEL REQUIREMENTS TO LOW-LEVEL SERVICES

The essence of the design phase lies in creating a bridge between the high-level requirements specified in the problem domain and the actual available resources. The high-level requirements or features are the realisation of use-cases, implementing services and operations of the application and its corresponding commands. On the other hand, the available resources are operating system, class libraries and the previously used application.

This jargon can easily be exemplified using a web-based ordering system. The web-based ordering system, as we all know, cannot be implemented using a programming language like C or a spreadsheet application. Even after the application has been designed on HTML/CSS or JavaScript or using other similar tools, it would need some intermediate resources like an IIS server(or some other web server) or some intermediate classes for either accommodating the network or handling the distributed nature of the application. By doing so, the designed application can run on the operating system and its hardware. These intermediate elements could also be elements of a framework or library. They are used to bridge the gap between the developed application and the available resources, as shown in Figure 10.1.

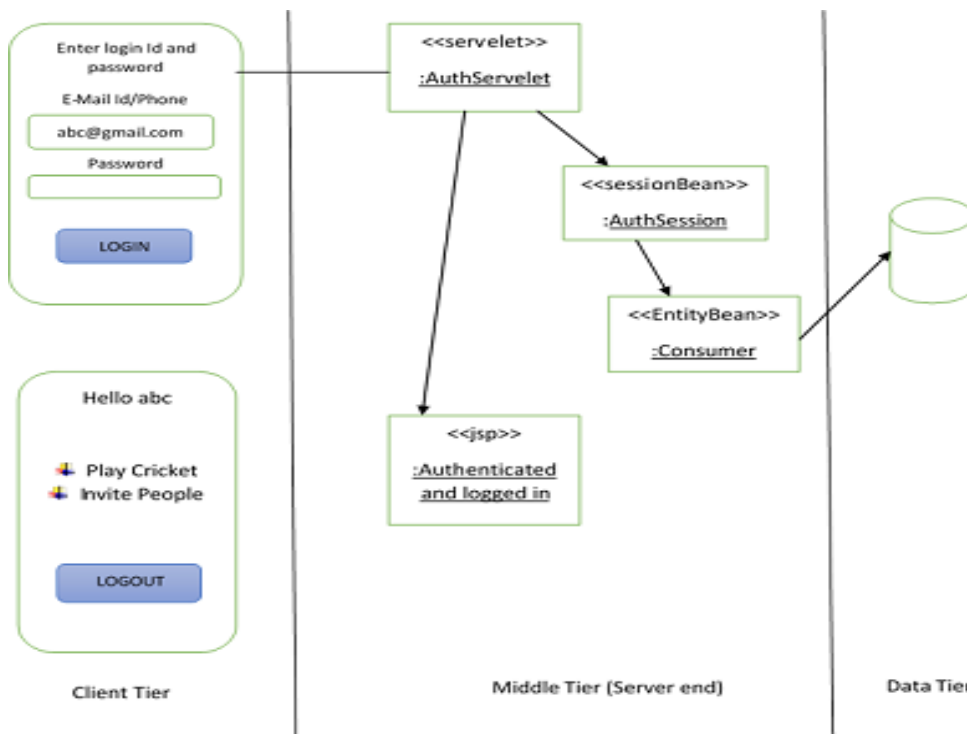


Figure 10.1: Mapping from high-level requirements to actual resources available

Design requires continuous synthesis; an ideal system cannot be formulated using the requirements only. The options of intermediate elements are so vast that it would not be viable to reach the right solution without applying heuristics.

10.5 REALISATION OF USE CASES USING OPERATIONS

While building the class diagram and communication diagram, different operations were defined to communicate between different objects. These operations were devised from use cases. Though the use case view is central to all views in object-oriented methodology as it articulates the business domain more closely. Hence,

during requirement phase, the behavior is specified but not its realisation. The realisation involves introduction of new objects and operations for including complexities of implementation environment, which could further be used for mapping to lower-level services.

A use case provides behaviour on the system level. The behaviour of a use case can also be thought of as a responsibility to be fulfilled. Responsibility of an object is defined as its ability to perform some action. For example, in an online theatre ticket booking system, 'Reserving a seat' is a use-case. This use-case uses objects to fulfil its responsibility by finding an unoccupied seat for the specific show, marking the seat allocated, getting payment from the customer and generating e-tickets. The theatre object handles this responsibility by tracking the filled seats, marking the prices for seat type, categorising the different seats and so on.

Each operation can handle many responsibilities. Some responsibilities may be shared whilst some are reused. Further, we assign these responsibilities to classes, and if these responsibilities cannot be included within the current classes, new classes need to be constructed.

Let us further consider an example of an ATM machine. The responsibility of the ATM Machine is to provide transactions. The transactions could be in the form of withdrawal from or deposit money or even changing the ATM PIN.

Example: Withdrawing money from ATM

This would involve several responsibilities:

- Reading the debit card.
- Asking amount from the customer.
- Check the account balance.
- Verify the transaction limit.
- Verify the sufficiency of cash in the ATM.
- Release money.
- Debit bank account.
- Print the receipt.
- Release the card.

We can see in this case that releasing money from the machine and debiting the bank account should happen one after another to maintain the consistency of the database.

Similarly, depositing money would again be a list of responsibilities:

- Getting the amount from the customer.
- Accept the amount.
- Timestamp the amount received.
- Credit bank account.
- Generate receipt for deposit.

The responsibilities of an object may have many operations involved. We are defining these complex operations while building class design. Our choice of responsibilities is such that it adheres to balancing performance, reliability, ease of future enhancement and other factors. In the next section, we will consider designing algorithms which help objects fulfil their responsibilities.

 Check your Progress-1

What are low-level services, and how are they linked to requirements?

.....

.....

.....

.....

What is a responsibility? Where do we get responsibilities from? With the help of a use-case describe steps to fulfil the responsibility.

.....

.....

.....

.....

List the steps to design objects?

.....

.....

.....

.....

10.6 DESIGNING ALGORITHMS

The signature of an operation created during analysis tells what the operation is going to perform but not the steps of its execution. Hence, the algorithms are formulated for each operation to show the steps. The steps for designing an algorithm should cater to the following conditions given below:

The choice of the algorithm requires the minimisation of cost.

Selection of data structures that are appropriate for the algorithms.

Mapping operations for new external classes and operations wherever necessary.

Assign the operations identified to appropriate classes.

The steps for designing are explained in the next sections.

10.6.1 Choosing Algorithms

Algorithms can be implemented in the simplest forms when efficiency is not an issue. It is a well-known fact that in a system, generally, 20% of the operations become bottlenecks. These operations spend 80% of the time on execution and implementation. Hence, focusing on the efficient design of these algorithms is important. The rest of the algorithms can be straightforward, understandable, and easy to program. The factors that impact the choice of algorithms are:

Computational complexity: The execution time significantly impacts the computational complexity. The algorithm designer should restrain the execution time from growing with input. The execution times could be constant, linear, quadratic or exponential. For example, algorithms like bubble sort have time complexity of the order of n^2 as compared with other algorithms and can be avoided.

Ease of implementation: While moving into the design model, the analysis domain class model should be negligibly customised. If the algorithm is inflicted with performance issues, only then it should be optimised. If many changes are made in the design model, it loses traceability and is difficult to understand.

Flexibility: Flexibility is a feature offered while creating algorithms. It can be offered if two versions of the same algorithm are provided. The algorithms that need to implement critical operations can have one version that is simple and quick to implement, although being inefficient. And then if there is an easy version that can be used to validate the system. The counterpart of the same algorithm can be an optimized, efficient, and complicated version. The complicated version can be authenticated with the simple one.

10.6.2 Selecting Data Structure

Algorithms work on data structures. In the design phase, we need to compose data structures in a manner such that the algorithms can run efficiently. These data structures could be generic data structures defined in class libraries, called container classes. These classes could be templates for arrays, lists, queues, stacks, dictionaries, variations of trees and many more.

10.6.3 Defining Internal Classes and Operations

During the analysis phase, operations were merely a way of organizing/realizing use cases. So operations were revealed as a side effect while modeling use-cases to support implementation during design. The UML term ‘operation’ maps to ‘message’ and ‘method’ in design. As we approach design, this terminology befits the programming language constructs such as message and method. Messages are added to classes as a result of:

Changing the value of fields since the object aspires to be read or changed.

The client object wants to use derived data.

The intuition or experience of the designer requires a particular message to be added as it might be required in the future.

Some frameworks or patterns being employed in the project require certain messages to be added.

So, this clearly indicates that mapping of attributes and relationships of analysis classes into their design counterparts would result in the extensive creation of messages.

10.6.4 Assigning Operations to classes

It is easy to assign operations to a class when it is a reflection of real-world scenarios. During design, new internal classes emerge that do not correspond with the aspects of the real-world scenario. It then becomes a challenge to find the correct class which is going to hold an operation. If only one object is involved in the operation, the decision process becomes easy, as the same object accomplishes the operation. The difficulty arises in cases where there are multiple objects, and a decision must be made to select the primary object. The following questions can lead us to a suitable class.

Receiver of action: Operations can best be associated with the target class rather than the class which initiated that operation. The object that has been modified using that particular operation is where it belongs to. Whether it is an update or a query, check for the object modified by the operation, not for those objects on which queries are made.

Focal class: It is a class in the class diagram which forms a center or a star to other classes and their associations. The center class is the target for the operation.

In case it is still difficult to find the target classes for the operations, it would be good to consider them as real-world objects. These real-world objects working can be examined through the way that they can be stimulated, collaborated, nudged, or changed to activate that operation.

Let us consider an example of how internal operations operate on transactions occurring on an ATM and then look for the respective classes.

- When the transaction (withdrawal) occurs, the ATM has to get an amount for withdrawal from the customer. Hence, a direct relationship of the operation ‘*getAmount*’ with the ‘*Customer*’ class (which gets updated) can be seen. Since here only ‘*Customer*’ class is involved, we can add this operation to the class.
- This amount must be verified with the customer’s account balance. Hence the operation ‘*verify_amount(amount)*’ is added to class ‘*Account*’ and not to ‘*Customer*’.
- The operation ‘*verify_amount(amount)*’ verifies that amount is eligible to be withdrawn according to the bank’s policies. Hence, ‘*verify_amount(amount)*’ operation also becomes part of ‘*Bank*’ class.
- The third verification is at the level of ATM, and we can have an operation ‘*ATM.verify_amount (amount)*’ which checks for sufficient cash in ATM.

- The ATM must *'disburse_funds(amount)'* to the customer. Hence the operation *'disburse_funds'* maps to the class *'ATM'*.
The bank account of the customer is debited with the amount. Hence operation *'debit'* maps to the *'Account'* class of the customer. The transaction should be added to the customer's receipt. The most obvious class for this operation seems to be *'Customer'*, but this receipt corresponds better to *'CashCard'* class because this allows more acceptable tracking of the ATM session.

10.7 DESIGN OPTIMIZATION

The design model is built after we add the implementation environment details in the analysis model. The analysis model gives a robust structure whose performance can be optimized for the inefficient constructs. But an optimized system might not be reusable and may be more confusing to understand. Hence, the act of balancing between efficiency and clarity becomes the biggest challenge for the designer. The designer tries to optimize performing tasks given below:

- Look for access paths that are efficient.
- Try to rearrange the computing tasks for greater efficiency.
- Save intermediate results for avoiding recomputation.

The subsequent sections describe these tasks in detail.

10.7.1 Look for efficient access paths

During analysis modeling, redundant associations are undesirable as they repeat information that already exists. Design modeling maps the analysis model to implementation; hence we might have to include redundant associations to improve the design. Questions like should there be new associations, what current associations can be omitted, or what associations have to be rearranged or might have to be proposed to address design solutions? Let us examine an analysis class model as given in Figure 10.2.

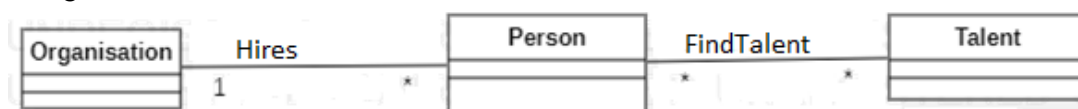


Figure 10.2: Analysis phase object model for finding a Talent in a Person in a company.

The operation *'organization.Findtalent()'* returns a set of employees in the organization with a given talent. For example, the management might need to know about employees knowing the Japanese language. Now if the algorithm runs for 100 employees, each of which has 10 people in the talent pool, on an average, there would be 1000 traversals for employees and 10,000 traversals for *'Findtalent()'*. In case there are only 5 employees who speak Japanese, then the test-to-hit ratio is 2000.

To improve the *'FindTalent()'* algorithm, the developer could implement it using a hashed set rather than an unordered list. With hashing, the search time always becomes constant. So, the cost of finding a person know Japanese in the talent pool becomes constant, provided a unique *'Talent'* object exists for the language. This rearrangement reduces the number of tests from 10,000 to 1000. Another option would be to use an index to retrieve frequently accessed objects. But indexes mandate additional memory and need updates whenever base associations change. Hence designer has to be careful in checking the utility of creating indexes.

10.7.2 Rearranging computing tasks for greater efficiency

Execution order of an algorithm needs to be optimized to improve the system's efficiency. Consider the previous example, suppose the application has to find all employees who speak both Japanese and Spanish. Out of 1000 employees, only five speak Japanese and 100 speak French; the algorithm's performance improves if the execution is performed first for Japanese speakers and then for French speakers. The search results are narrowed down, and the outcome can be achieved quickly.

10.7.3 Saving intermediate results for avoiding recomputation

A derived attribute is a function of one or more elements. For example, in a bank 'Account' class, the balance is a derived attribute computed using 'CurrentAmount' and the 'AmountWithdrawn'. It is dependent on the constraint that the amount withdrawn cannot be greater than the current amount.



{balance = CurrentAmount – AmountWithdrawn}

Figure 10.3: Derived Attribute is shown with a '/' in front.

The derived element is shown with a slash specified ahead of the element name, as given in figure 10.3. The class model differentiates between the independent base attributes and dependent derived attributes. These derived attributes might be added to new classes to store their values and avoid recomputation. When the base class is updated, the classes containing the derived attributes are also updated. The updations can be carried out in three different ways, as given below.

Explicit update: The updation operations of the base class contain code that explicitly updates derived attributes dependent on it.

Periodic recomputation: Applications tend to update values of attributes in bunches. Periodically recomputing derived attributes makes them less prone to bugs. Hence instead of single updates, periodic updates could be a better option.

Active values: Some attributes need to be kept consistent with their base attributes as they form part of a monitoring mechanism. A special registration mechanism keeps track of these updated source attributes and consequently reflects the updations in derived attributes.

ATM example: A bank looking for tracking or monitoring suspicious customers. A customer is considered to be suspicious by a bank when he/she attempts to evade the limits defined by the bank. So, a class 'Suspicious_Group' could have numerous updates. The new class 'Suspicious_Group' caches the derived attribute so that suspicious behaviour can be monitored. Also, it would be convenient to record comments and annotations in this class.

Check your Progress-2

What are the metrics for choosing the best algorithms?

.....

.....

.....

.....

What are derived attributes? What role do they play in design optimization?

How do we assign operations to classes?

10.8 IMPLEMENTATION OF CONTROL

As you know, the controller class manages the controls between different objects. The class aptly receive signals from the external environment or from the objects interacting inside the system. The reaction of the class to these signals essentially results in the operation on objects in the system and transmitting these signals to the outer world.

So, a controller class can be thought of as a piece of reified behaviour captured within an object so that it can be transformed more easily. While implementing design, these classes are introduced to capture the behaviour that does not belong to a particular object, and most applications will have these controller classes to sequence its behaviour. So, to implement these classes, we look for classes that have multiple events and use the timing model to find events for those classes. Then the relevant events are organized in a sequence so that a state transition diagram can be prepared for the class.

The consistency of state diagrams is validated against classes and sequence diagrams while building the application. To implement an applications state model, the following steps are performed:

- 1) Find the respective classes for states.
- 2) Find the triggering events.
- 3) Merge state diagrams into code.
- 4) Examining against other state diagrams.
- 5) Examining against other class diagrams.
- 6) Examining against sequence diagrams.

These steps are explained in detail in the subsequent sections.

10.8.1 Find the respective classes for states

The prominent classes to the application are implemented, and corresponding users are also identified. Among these classes, we need to identify those with multiple states. User interface and controller classes are a good place to start for identifying the states. Normally entity classes are used for storing information and being static in nature, and they do not have multiple states.

10.8.2 Find the triggering event

Explore the scenarios discussed in the use cases to identify the events. They are a good source for identifying the common interactions and major events. While working with the application state model, identification of events is sequenced before determining the states as the emphasis is on behaviour.

10.8.3 Merge state diagrams into code

After identifying the events, the focus is now on constructing a state diagram for each application class. The classes which have a temporal behaviour are selected. To start with,

- Select one of the temporal classes and a sequence diagram which contains that class.
- The events of the class are then arranged into a path whose arcs are labelled by the name of the events.
- The arcs are connected using states/events.
- After merging one sequence diagram into a state diagram, select another sequence diagram containing the class selected earlier and perform step 2 and 3 again to make the complete state diagram.
- Look for repetitive loops or repetitive sequences that can be unambiguously identified by the same first and last state.
- Loops should always have a terminating state that can be categorized with multiple transactions leaving it.
- Alternate paths can be detected from sequence diagrams. Look for a point in the sequence diagram which diverges from the previous one; this point indicates the existence of a state in the existing diagram. The new sequence of events for an existing state are added as an alternative path in the state chart diagram.
- The alternate paths created because of new events have to be re-joined to an existing state in the diagram. Two paths can be joined at a state if an object “forgets” which path was traversed.
- There could be two paths which appear identical, but in some instances, those might be disparate. For example, when the password is entered on an ATM machine, it can have a path where the account is verified, and the alternative path would be that a password is entered incorrectly. The alternative path again has two distinguishing paths, one which asks for the password again and the other one being that if three attempts have occurred, the card gets blocked.
- Consider adding variations and exceptional events after a normal case has been accounted. For example, in an ATM, if a cancellation request is issued just immediately after submitting it or the user might not respond to the instructions on the machine.

A good designer, after completing these steps, will validate the diagram examining whether all events and scenarios have been covered. This can further lead the designers to suggest some events that have not been handled and can impact the state. Consider the state chart diagram building for placing the order on the website. The activity diagram in figure 10.4 shows the steps for placing an order on the website. It is clear from the diagram that the order class is having different states while the order is being processed, as shown in figure 10.5.

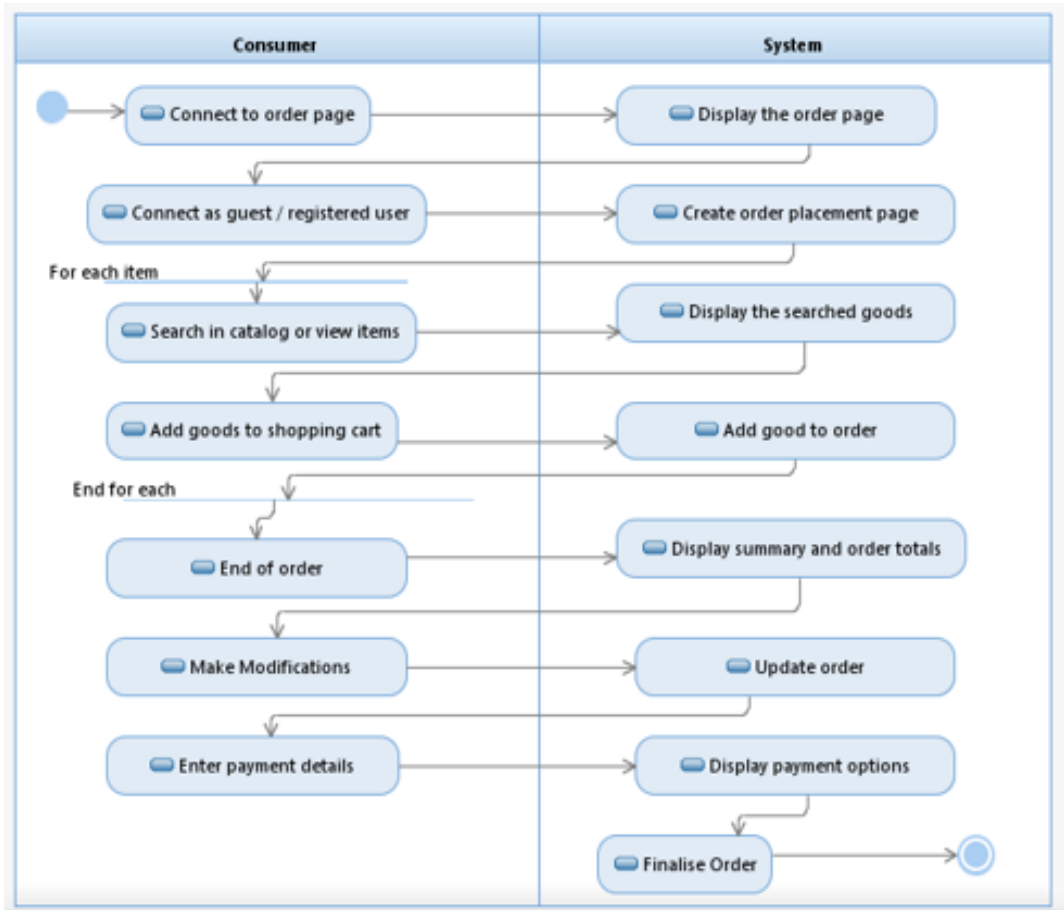


Figure 10.4: Activity Diagram for placing a new order.

The object transition takes place from one state to another as messages are passed to the different states, as shown in table 10.1.

Table 1: State and Exit Transitions for Order class

STATE	TRANSITION CAUSING EXIT FROM STATE
Open and add items	OrderComplete
Item ready to be shipped	StartShipping
BeingShipped	CurrentShipping
Waiting for back orders	Arrivalbackorders
Shipped	Payment_clearance
Closed	No exit transition required

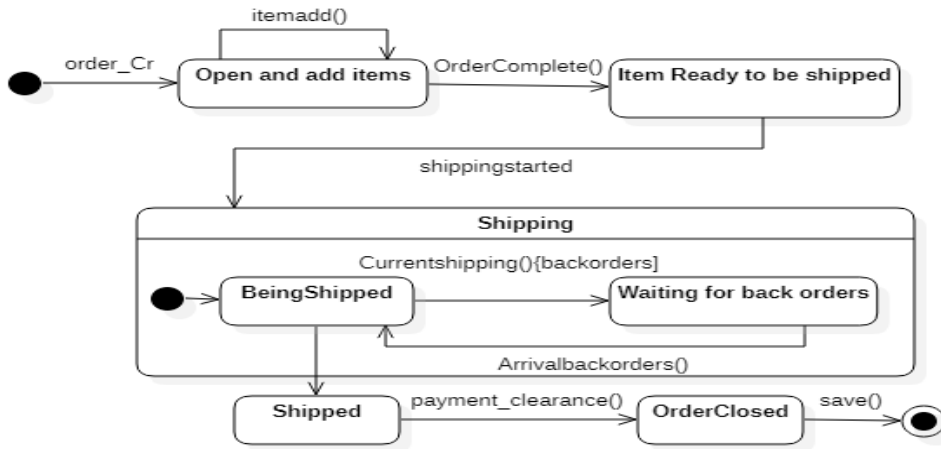


Figure 10.5: First cut state transition graph for Order Class.

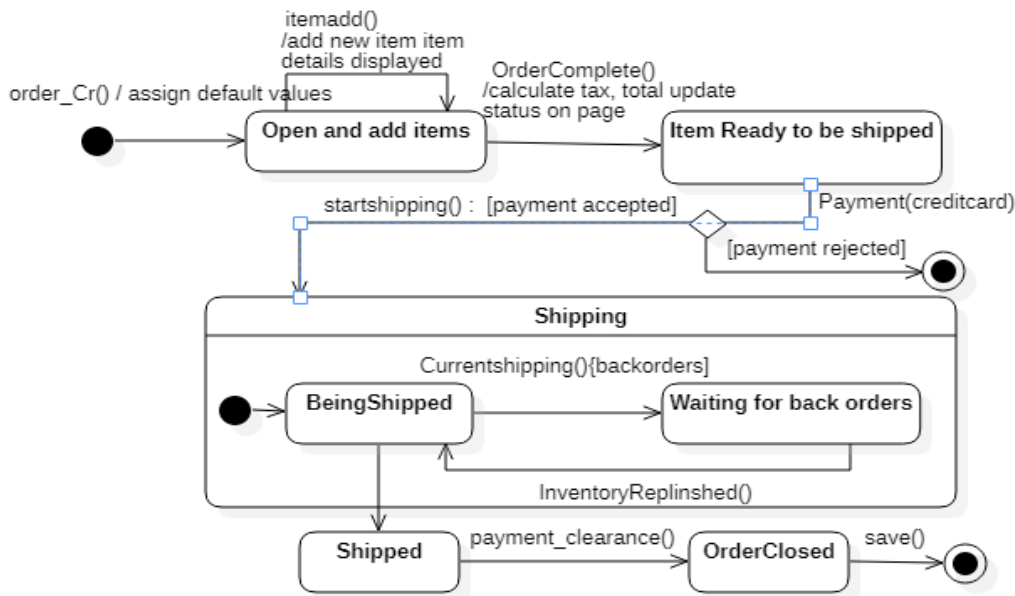


Figure 10.6: Design state diagram for Order Class

The state transition graph is shown in figure 10.6.

10.8.4 Examining against other state diagrams

The state diagrams for each class can now be examined for their consistency and completeness.

- Examine states for having both sender and receiver messages.
- Examine states for their predecessors and successors along with starting and ending points.
- Examine the input event sequence from object to object so that the scenarios they represent are validated.

10.8.5 Examining against class model

The state diagrams need to comply with their domain and application class models. For example, it is possible that many ATMs can concurrently try to access an account, but it needs to be ensured that only one update is applied at a particular time. The consistency of the account has to be maintained.

10.8.6 Examining against sequence diagram

The compliance of the state model with scenarios of timing diagrams has to be examined. Simulation of behaviours must be performed with hands to verify the correctness. In case of an error, modification in the state diagram or scenario would help remove irregularities. These irregularities could be in both either in the diagrams or the scenarios. Tracing out legitimate paths of the state model can bring into picture additional events which might or might not make sense and hence need to be examined closely.

10.9 ADJUSTMENT OF INHERITANCE

It would be convenient to create inheritance hierarchies between classes as we proceed further with the design. The following steps guide understanding the adjustment of classes and operations.

- Rearrangement of classes and operations to augment inheritance.
- Abstractions of common behaviour from a group of classes

10.9.1 Rearranging Classes and Operations

When we examine classes during design, we find that several classes define the same operations. In such a case, creating an inheritance hierarchy using common ancestors is common practice. Though, often, those operations in different classes are not identical. Hence, to include them within a single inheritance, the definition of the operation must be adjusted as per need.

The similarity between operations can be adjusted to make them identical by making their signatures the same and adjusting the number and type of arguments and outcomes. There are four different ways of adjusting the different types of operations to improve the chances of inheritance.

Operations with optional arguments: Alignment of signatures is possible through optional arguments. The optional arguments can be ignored where they are not used. For example, drawing operation on a black and white display screen can add a color parameter to it as a signature for creating the alignment. But the drawing operation can ignore or accept the parameter depending on the type of display. In the case of black and white displays, we can ignore the given color parameter or argument.

Operations that are special cases: Some operations that have a smaller number of arguments are normally special cases of general operations. Such operations can be adjusted by using appropriate values for parameters in the generic operations. For example, inserting an element in a list is a generic operation and appending an element becomes a special case where the element is added at the end.

Generalization of names: Attributes having different names in different classes must be homogenized. These attributes which have been homogenized will have the same nomenclature. They can now be moved to parent classes. Operations that access those attributes will also match appropriately. Just as attribute names have been homogenized, even operation names need to be homogenized.

Irrelevant operations: Consider the classes in a group; some of them might have elaborate definitions for operations, and other classes might not have any operation. Such operations can be moved to parent classes, and while

declaring these operations, they can be specified as ‘no_op’ for those classes which do not use them.

10.9.2 Abstraction of Common Behavior

While adding new classes and operations during design, the opportunity of adding inheritance hierarchy can also be explored. An inheritance hierarchy is easily recognisable amongst two classes with repetitive operations and attributes. They could possibly be specializations of the same thing giving a higher level of abstraction.

As is in every inheritance relationship, a common superclass is created in which the common features are abstracted out, and specialized features are defined within subclasses. Hence, abstractions can be defined. However, it is a good practice not to create direct instances of super classes. The ‘draw()’ operation turns out to be a good example of this relationship as the draw method could be used for rendering circles, squares, and lines in the specialized classes but attributes like thickness, color of the figure, and type of line are elements of the abstract class, ‘*Shape*’.

Inheritance is a valuable proposition for future projects. Even though looking at the current scenario, we can see that multiple subclasses are not an immediate requirement. As in the current classes there is no probability of sharing behaviour. But while designing, potentially reusable classes for future projects should be considered.

Inheritance promotes modularity as specific aspects are separated from general aspects in inheritance-based classes. The creation of abstract classes allows the software to be extended in different frameworks. Imagine that you are developing a gas leakage sensing module for a large factory so that leakage can be detected quickly and there are no fatalities. For that purpose, the module designed for one factory requires the detection of methane or butane gas which can be detected using a specific sensor, say MQ-2.

The module for gas leakage detection could use a single class, with one instance for each sensor used in the factory. However, if we keep in view the future scope of the project, it might be possible that there would be other factories looking for similar deployment. But this deployment would require other types of gases to be detected through different sensors. So, to meet this requirement, it would be a good idea to create a superclass ‘SensorNode’ which contains the common behaviour of sensors and a subclass SensorMQ2, which provides reading and detection particular to sensor, MQ-2.

Now, the leakage detector system of a factory is deployed in another factory which requires natural gas LPG detection. We can simply create a new subclass sensorMQ5, and specialized behaviour can be added to this class without any changes to the existing classes, as shown in figure 10.7.

Another aspect which improves with abstraction is the maintenance of software. Suppose this software is being distributed to many units of the same plants, where the configuration is different for every unit. Imagine the kind of complexity that would have been introduced if a customized version had to be created for every different sensor. But this complexity could easily be handled if one software version contains a subclass for each model. After deployment, the software inspects the configuration file and creates instances of particular subclasses for sensors available in the corresponding plant.

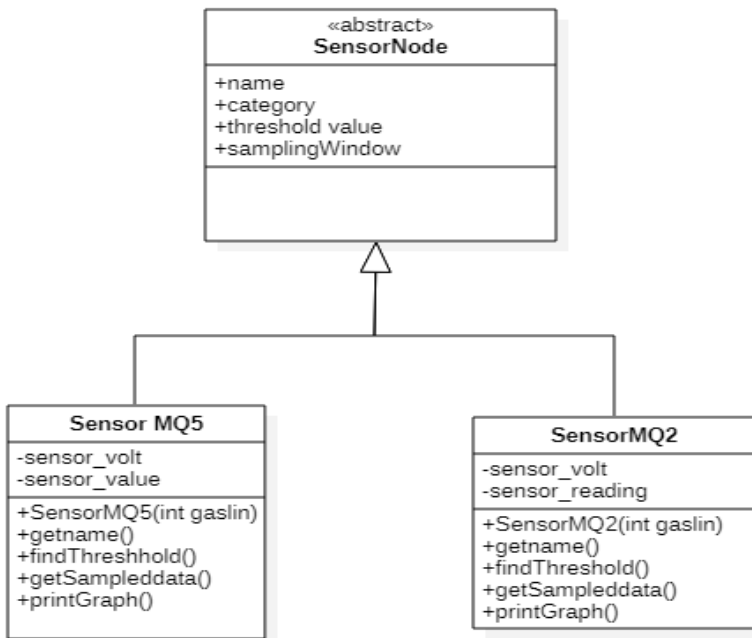


Figure 10.7: Abstraction of common behaviour

10.10 DESIGN OF ASSOCIATIONS

As it is known, the diagrammatic representation of classes in UML is differentiated from its instances using an underline below their names. While progressing towards design, there are three major issues related to associations that must be determined.

- Identifying classes to be implemented
- Identifying types of attributes
- Mapping composition relationships

Decisions related to retaining classes identified during the analysis phase must be made, along with adding any other supporting classes. The attributes of classes can be rendered as private fields or members of the class. In the case of an optional attribute, the only change that occurs is at runtime; that is, the corresponding field might take a null value.

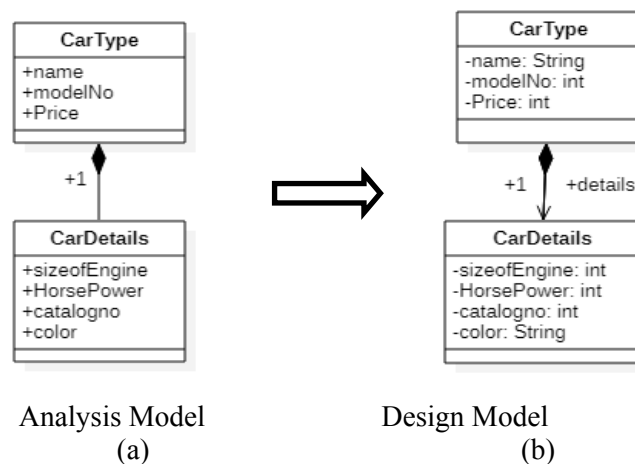


Figure 10.8: Representation of attributes and compositions from analysis model into the design model

In the design phase, the composition relationship has to be reconsidered. When we analyzed our problem, having the composition relationship, we found that the

relationship directions allow searching either from one end, or the direction is bidirectional. But with design, restrictions start to appear on fields, allowing interaction in one direction only (for example, at runtime, the owning object can access the object at the other end but not in the reverse direction). So, a decision must be made whether the relationship can be extended at each end or not.

As shown in figure 10.8(a), while building the analysis model, a composition's primary relationship starts from the composer (CarType class) to the composed (CarDetails class). The owner (CarType class) fetches the information from the composed object, but not the other way. However, it is possible that the design requires that the field (CarDetails) be embedded in both the composer and composed object, then that is also permissible while doing design. This mapping is done in a design class by adding an arrowhead in the class diagram along with the rule and visibility, as shown in figure 10.8(b). So, during execution time, the messages flow in the direction of the arrow.

10.10.1 Analyzing Association Traversals

There are three different kinds of associations: association (plain), aggregation (stronger) and composition (strongest). Object oriented languages do not have provisions to make any distinctions between aggregation and composition and hence consider all of them to be associations.

Most of the associations that have been identified in the analysis phase and also the new ones end up as fields in objects. But fields in a class permit navigation in one direction. Hence in the design phase, decisions related to navigation have to be made. These decisions can be dependent on the multiplicities, one-to-one, one-to-many and many-to-many relationships. Let us examine the associations one-to-one and one-to-many.

10.10.1.1 One-way Associations

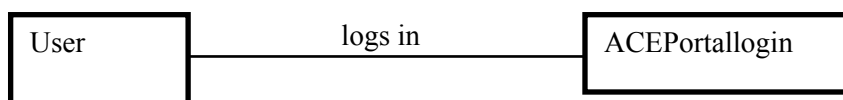


Figure 10.9: Class Diagram

If we look at the fragment of the analysis domain model diagram in figure 10.9, there is a one-to-one association. This association can be implemented in a programming language with three different possibilities.

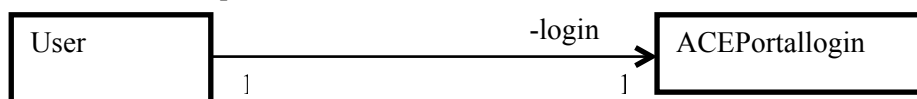


Figure 10.10: First alternative One-to-one association representation in a design class diagram

The first way to implement the one-to-one association is to take the 'login' association and add it as a field in 'User' pointing to 'ACEPortallogin' as in figure 10.10. The second way could be implementing 'login' as a field into the 'ACEPortallogin' pointing to the 'User' as given in Figure 10.11.

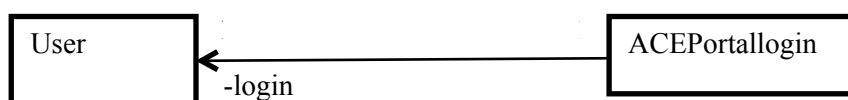


Figure 10.11: Second Alternative one-to-one association representation in a design class diagram

The third alternative would be combining the two ways shown above in Figures 10.5 and 10.6 to implement the two-way association. That is, both classes are going to use a pointer field to point to the other class. This would require a lot of synchronization, which is awkward to implement but possible. For example, the user class could contain a *'ACEPortallogin'* field pointing to the *'ACEPortallogin'* class. Similarly, the *'ACEPortallogin'* would contain a user field pointing to the *'User'* class. Any changes in the field would also require updations to the original class. It is always a natural option to choose the one-way alternative rather than linking the fields in both classes.

An optional association with a multiplicity of 0..1 is similar to 1..1 case, accepting that fields can be assigned null values (meaning no object at the far end).

10.10.1.2 One to many Associations

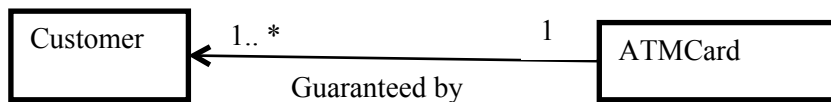


Figure 10.12: One-to-many association in a design class diagram

Mapping a one-to-many association to classes also uses the same arguments as for one-to-one cases. The difference is that if a field in the multiplicity at the '1' end is added to the other class, it must store multiple objects. So, for example, if a customer field is added to *'ATMCard'*, there would be multiple customer objects. Hence the *'ATMCard'* requires a collection class to hold the customer object. A list or array could be used. Depending on the availability of the construct in the language, Set or Bag could also be used, as shown in figure 10.13.



Figure 10.13: One-to-many association implemented as List in a design class diagram.

☛ Check your Progress-3

1. What are two-way associations? What are different ways to implement two-way associations while you are implementing the design model?

.....

.....

.....

2. How does the rearrangement of classes and operations augment inheritance?

.....

.....

.....

3. What are the steps to implement an application state model?

.....

.....

.....

10.11 SUMMARY

This unit covered class design implemented as an elaboration of the earlier phases. While performing class design, we add details related to designing algorithms, refactoring operations, class optimization, and adjusting inheritance hierarchies.

The first step involved enhancing the classes in the design phase by adding operations identified from use cases. New operations might also be added to deliver the new behaviours discovered. Hence use cases are realized during the design phase. After identifying operations, we focussed on algorithms for operations. Clarity of algorithms can be improved if a small amount of performance issues are sacrificed. The initial design of a set of operations is expected to be inefficient since it is difficult to obtain a large design correct in one pass. It requires iterations and refactoring of operations to improve resilience and clarity.

Defining abstract superclasses for retaining the common behaviours and subclasses to assimilate the specialized attributes and operations. As class design progresses, classes have the same common features found, and these classes now need to be adjusted and modified to create inheritance relationships between them. Associations between classes are also mapped to their corresponding classes as attributes or pointers.

10.12 SOLUTIONS/ANSWERS TO CHECK YOUR PROGRESS

Check Your Progress -1

1. Low-level services include operating system, plug-ins and class libraries (API, system calls). They are linked to high-level requirements since the high-level requirements are built using these low-level services. These requirements are required for building that application.

2. An object's responsibility includes knowing about the object's data and other classes and performing activities to execute the use case. We get responsibilities from use-cases. For example, a use-case may have the responsibility of registering a student will:

Step 1: Display the registration form.

Step 2: Fill in the details. All details are checked for their validity.

Step 3: Save the details.

3. The steps to designing objects are:

- creating links between high-level requirements to low-level services,
- the realization of use-cases using operations,
- formulating an algorithm,
- designing operations that assist high-level methods,
- refactoring for a clean design,
- optimization of data access paths,
- realising behaviour that must be deployed,
- adjustment of classes to stipulate inheritance and,
- organize classes with their associations.

Check Your Progress 2

1. The metrics for choosing the best algorithms are efficiency, computational complexity, ease of implementation and flexibility.

2. A derived attribute is a function of one or more elements. For example, we have two attributes – date of birth and today's date, and the age becomes a derived attribute

as it can be calculated from these two existing attributes. But since age is a dependent attribute, it can impact design optimization. In case these derived attributes need to be stored for periodic recomputation, new classes might be added.

3. During design, new internal classes emerge that do not correspond with the aspects of the real-world scenario. It then becomes a challenge to find the correct class which is going to hold an operation. If only one object is involved in operation, the decision process becomes easy, as the same object accomplishes the operation. The difficulty arises in cases where there are multiple objects, and a decision needs to be made to select the primary object. In case of multiple classes, the classes which receive an action can be assigned that operation. The other approach would be to look for focal classes.

Check Your Progress 3

1. Two-way associations are bidirectional associations. The implementation depends on the multiplicity defined. If the multiplicity is one-to-one, then a pointer to the reference class can be created using the programming language. The second way is to create a field pointing to the referred class. The last way would be that both classes have a pointer field to point to the other class.

2. When we examine classes during design, we find that several classes define the same operations. In such a case, creating an inheritance hierarchy using common ancestors is common practice. Though, often, those operations in different classes are not identical. Hence, to include them within a single inheritance, the operations definition must be adjusted.

The similarity between operations can be adjusted to make them identical by making their signatures the same, by adjusting the number and type of arguments and outcomes.

3. To implement an application state model, the following steps are performed:

- Find the respective classes for states.
- Find the triggering events.
- Merge state diagrams into code.
- Examining against other state diagrams.
- Examining against other class diagrams.
- Examining against sequence diagrams.

10.13 REFERENCES/FURTHER READINGS

1. Grady Booch, James Rumbaugh and Ivar Jacobson, “The Unified Modeling Language User Guide”, 2nd Edition, Addison-Wesley Object Technology Series, 2005.
2. Grady Booch, Robert A. Maksimchuk, Michael W. Engle, Bobbi J. Young, Jim Conallen, Kelli A. Houston, “Object-Oriented Analysis and Design with Applications,” 3rd Edition, Addison-Wesley, 2007.
3. Blaha, M., “Object-Oriented Modeling and Design with UML: For VTU, 2/e”, Pearson Education India, 2005.
4. O'docherty, Mike, “Object-oriented analysis & design”, John Wiley & Sons, 2005.